

AWS Deployment Guide

Agentic Compliance Auditor

Architecture · Code changes · Step-by-step setup

Prepared for: self-hosted deployment of a RAG pipeline using FastAPI, ChromaDB, Redis, and Ollama on Amazon Web Services.

Contents

- 1. Architecture overview
- 2. Deployment options (EC2, ECS, EKS)
- 3. Pre-deployment code changes
- 4. Step-by-step EC2 deployment
- 5. Containerized deployment (Docker + ECS)
- 6. Switching Ollama to Amazon Bedrock
- 7. Managed services: ElastiCache, OpenSearch, S3
- 8. Security hardening
- 9. Monitoring & observability
- 10. Cost estimates
- 11. Troubleshooting checklist

1. Architecture overview

The application consists of four logical tiers:

- **UI tier** — Streamlit (*ui/streamlit_app.py*), calls the API over HTTP.
- **API tier** — FastAPI (*src/api/main.py*), handles queries, memory, ingestion.
- **Retrieval / agent tier** — ChromaDB + LangGraph agent running in-process with the API. Uses Ollama for LLM inference by default.
- **State tier** — Redis (session memory) and the ChromaDB persistent directory.

For AWS, each tier maps to a managed or compute service. The simplest path is a single EC2 instance running everything; the scalable path separates the LLM, vector store, and API into dedicated services.

Tier-to-service mapping

Tier	Recommended AWS service
UI (Streamlit)	EC2 / App Runner / Amplify (behind CloudFront)
API (FastAPI)	EC2 / ECS Fargate / App Runner
LLM inference	Amazon Bedrock (recommended) or Ollama on EC2 g5
Vector store	ChromaDB on EBS, or Amazon OpenSearch w/ k-NN
Session memory	Amazon ElastiCache for Redis
Document storage	Amazon S3 (raw docs), EBS (processed DB)
Secrets	AWS Secrets Manager / SSM Parameter Store
Logs & metrics	CloudWatch Logs, CloudWatch Metrics

2. Deployment options

Three viable paths, in order of complexity. Pick based on the audience and load.

Option A — Single EC2 instance (recommended for demo/PoC)

- One EC2 instance runs FastAPI, Streamlit, Ollama, Redis, and ChromaDB (files on EBS).
- **Pros:** Simplest setup, lowest cost, one box to debug.
- **Cons:** No HA, LLM on CPU is slow; upgrade to *g5.xlarge* for GPU acceleration.
- **Instance:** *g5.xlarge* (1x A10G, 16 GB GPU) for real-time inference, or *c6i.2xlarge* for CPU-only.

Option B — ECS Fargate for API + managed AI services

- API and UI run as Fargate tasks behind an Application Load Balancer.
- LLM is served by Amazon Bedrock (no self-hosted Ollama).
- Vector store moves to OpenSearch Serverless (k-NN).
- Redis via ElastiCache, documents in S3.
- **Pros:** Fully managed, auto-scaling, no GPU to manage.
- **Cons:** More services to configure, higher baseline cost.

Option C — EKS (Kubernetes)

- Only if you already run Kubernetes. Overkill for this project.
- Same components as Option B but packaged as pods and deployments.

The rest of this guide details Option A end-to-end, then shows how to migrate individual pieces to managed services (Option B).

3. Pre-deployment code changes

The codebase already works on localhost. These changes make it production-ready before you ship the image or bake the AMI.

3.1 Use *absolute paths anchored to the project root*

Already applied in config/config.py: CHROMA_PERSIST_DIR, LOG_FILE, and EVAL_DATASET_PATH resolve relative to the config file so the app works from any working directory. Keep this.

3.2 Replace *@app.on_event* with *lifespan*

The FastAPI deprecation warning becomes a hard error in newer versions. Update `src/api/main.py`:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # --- startup ---
    global vector_store, retriever, agent, memory
    vector_store = VectorStore(
        persist_directory=settings.CHROMA_PERSIST_DIR,
        collection_name="compliance_documents",
        embedding_model=settings.EMBEDDING_MODEL,
    )
    retriever = CitationRetriever(vector_store, top_k=settings.TOP_K_RESULTS)
    tools = create_langchain_tools(retriever)
    agent = ComplianceAgent(retriever, tools,
                           model_name=settings.OLLAMA_MODEL,
                           max_iterations=settings.MAX_ITERATIONS,
                           enable_reflection=settings.ENABLE_SELF_REFLECTION)

    try:
        memory = RedisMemory(host=settings.REDIS_HOST,
                             port=settings.REDIS_PORT,
                             db=settings.REDIS_DB)
    except Exception as e:
        logger.warning(f"Redis unavailable ({e}); using in-memory buffer")
        memory = ConversationBufferMemory()
    yield
    # --- shutdown ---
    logger.info("API shutting down cleanly")

app = FastAPI(title="Agentic Compliance Auditor API",
              version="1.0.0", lifespan=lifespan)
```

3.3 Lock down *CORS*

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.environ.get("ALLOWED_ORIGIN", "https://your-ui-domain.example.com")],
    allow_credentials=True,
    allow_methods=["GET", "POST", "DELETE"],
    allow_headers=["*"],
)
```

3.4 Remove *reload=True* and *bind workers* from *settings*

```
if __name__ == "__main__":
    import uvicorn
```

```

uvicorn.run(
    "main:app",
    host=settings.API_HOST,
    port=settings.API_PORT,
    workers=settings.API_WORKERS,      # 2-4 per vCPU
    log_level=settings.LOG_LEVEL.lower(),
)

```

3.5 Secure the /ingest endpoint

The current endpoint accepts an arbitrary `file_path` query parameter and reads from disk. Before deployment, either remove the endpoint or require an authenticated S3 URI:

```

@app.post("/ingest")
async def ingest_document(s3_uri: str, bg: BackgroundTasks,
                          token: str = Depends(verify_api_token)):
    assert s3_uri.startswith("s3://"), "Only s3:// URIs allowed"
    bg.add_task(ingest_from_s3, s3_uri)
    return {"status": "queued", "s3_uri": s3_uri}

```

3.6 Add API-key authentication

```

from fastapi import Depends, Header, HTTPException
import os, secrets

API_TOKEN = os.environ["API_TOKEN"]

def verify_api_token(authorization: str = Header(...)):
    if not authorization.startswith("Bearer "):
        raise HTTPException(401, "Missing Bearer token")
    if not secrets.compare_digest(authorization.split()[1], API_TOKEN):
        raise HTTPException(403, "Invalid token")
    return True

# Add Depends(verify_api_token) to every POST/DELETE route.

```

3.7 Structured logging to stdout

CloudWatch Logs captures stdout/stderr from EC2 (via agent) and Fargate (automatically). Configure loguru:

```

from loguru import logger
import sys, json

logger.remove()
logger.add(sys.stdout, serialize=True, level=os.environ.get("LOG_LEVEL", "INFO"))

```

3.8 Health and readiness endpoints

```

@app.get("/healthz") # liveness – always 200 if process alive
async def healthz(): return {"ok": True}

@app.get("/readyz") # readiness – checks dependencies
async def readyz():
    try:
        assert vector_store.get_collection_stats()["total_chunks"] > 0
        if isinstance(memory, RedisMemory):
            memory.client.ping()
        return {"ready": True}
    except Exception as e:
        raise HTTPException(503, str(e))

```

4. Step-by-step EC2 deployment (Option A)

4.1 Provision the instance

- Region: pick the closest to your users (e.g. *us-east-1*).
- AMI: Ubuntu 22.04 LTS (ami-0xxx).
- Instance type: **g5.xlarge** for GPU Ollama, or **c6i.2xlarge** for CPU-only (plus Bedrock for LLM).
- Storage: 100 GB gp3 EBS. Increase if documents > 5 GB.
- Security group: inbound 22 (SSH from your IP), 8000 (API, from ALB SG only), 8501 (UI, from your IP / VPN / CloudFront only).
- IAM role: attach a role with S3 read, Bedrock:InvokeModel, Secrets Manager read, and CloudWatch Logs write.

4.2 Base setup

```
# SSH in
ssh ubuntu@<PUBLIC_IP>

sudo apt update && sudo apt -y upgrade
sudo apt install -y python3.11 python3.11-venv python3-pip git redis-server nginx
sudo systemctl enable --now redis-server

# Verify
redis-cli ping    # -> PONG
```

4.3 Install Ollama (skip if using Bedrock)

```
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &                # starts the daemon
ollama pull llama3.2           # ~2 GB
```

4.4 Clone and configure the app

```
cd /opt
sudo git clone <YOUR_REPO_URL> compliance-auditor
sudo chown -R ubuntu:ubuntu compliance-auditor
cd compliance-auditor

python3.11 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install gunicorn
```

4.5 Create the production .env

Fetch secrets from Secrets Manager rather than hard-coding. Example *.env*:

```
# LLM
OLLAMA_BASE_URL=http://127.0.0.1:11434
OLLAMA_MODEL=llama3.2

# Vector DB
EMBEDDING_MODEL=all-mpnet-base-v2

# Redis (local)
```

```

REDIS_HOST=127.0.0.1
REDIS_PORT=6379
REDIS_DB=0

# API
API_HOST=0.0.0.0
API_PORT=8000
API_WORKERS=4

# Agent
MAX_ITERATIONS=3
ENABLE_SELF_REFLECTION=false

# Auth & logging
LOG_LEVEL=INFO
API_TOKEN=<generate-with-openssl-rand-hex-32>
ALLOWED_ORIGIN=https://your-ui-domain.example.com

```

4.6 Ingest documents

```

source venv/bin/activate
# Pull raw docs from S3
aws s3 sync s3://your-bucket/regulations ./data/sample_docs

python -c "
import sys; sys.path.insert(0,'src'); sys.path.insert(0,'.')
from config import settings
from retrieval.vector_store import VectorStore
from ingestion.document_loader import DocumentLoader
from ingestion.chunker import RegulationChunker
vs=VectorStore(settings.CHROMA_PERSIST_DIR, 'compliance_documents',
               embedding_model=settings.EMBEDDING_MODEL)
loader=DocumentLoader(); chunker=RegulationChunker(512,0)
docs=loader.load_directory('./data/sample_docs')
chunks=[c for d in docs for c in chunker.chunk_document(d)]
vs.add_documents(chunks); print(vs.get_collection_stats())
"

```

4.7 Run FastAPI with Gunicorn + systemd

```

# /etc/systemd/system/compliance-api.service
[Unit]
Description=Compliance Auditor API
After=network.target redis-server.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/compliance-auditor
EnvironmentFile=/opt/compliance-auditor/.env
ExecStart=/opt/compliance-auditor/venv/bin/gunicorn \
    -k uvicorn.workers.UvicornWorker \
    -w 4 -b 0.0.0.0:8000 \
    --chdir /opt/compliance-auditor/src/api main:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload

```



```
sudo systemctl enable --now compliance-api
sudo systemctl status compliance-api
curl http://127.0.0.1:8000/healthz
```

4.8 Run Streamlit UI as a service

```
# /etc/systemd/system/compliance-ui.service
[Unit]
Description=Compliance Auditor UI
After=network.target compliance-api.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/compliance-auditor
EnvironmentFile=/opt/compliance-auditor/.env
ExecStart=/opt/compliance-auditor/venv/bin/streamlit run ui/streamlit_app.py \
    --server.port 8501 --server.address 0.0.0.0 --server.headless true
Restart=always

[Install]
WantedBy=multi-user.target
```

4.9 TLS + reverse proxy with nginx + Let's Encrypt

```
sudo apt install -y certbot python3-certbot-nginx

# /etc/nginx/sites-available/auditor
server {
    listen 80;
    server_name auditor.example.com;
    location / { proxy_pass http://127.0.0.1:8501; }      # UI
    location /api/ { proxy_pass http://127.0.0.1:8000/; } # API
}

sudo ln -s /etc/nginx/sites-available/auditor /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d auditor.example.com
```

5. Containerized deployment (Docker + ECS)

5.1 Dockerfile for the API

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt gunicorn
COPY . .
ENV PYTHONPATH=/app/src:/app
EXPOSE 8000
CMD ["gunicorn", "-k", "uvicorn.workers.UvicornWorker", "-w", "4",
    "-b", "0.0.0.0:8000", "--chdir", "src/api", "main:app"]
```

5.2 Build & push to ECR

```
aws ecr create-repository --repository-name compliance-auditor-api
aws ecr get-login-password | docker login --username AWS \
    --password-stdin <acct>.dkr.ecr.us-east-1.amazonaws.com
docker build -t compliance-auditor-api .
docker tag compliance-auditor-api:latest \
    <acct>.dkr.ecr.us-east-1.amazonaws.com/compliance-auditor-api:latest
docker push <acct>.dkr.ecr.us-east-1.amazonaws.com/compliance-auditor-api:latest
```

5.3 ECS task definition (key fields)

- **launchType**: FARGATE
- **cpu/memory**: 2 vCPU / 8 GB (API-only, with Bedrock for LLM)
- **environment**: REDIS_HOST, API_TOKEN, ALLOWED_ORIGIN, etc. (pull secrets from Secrets Manager via *secrets* block)
- **logConfiguration**: awslogs driver → /ecs/compliance-auditor
- **portMappings**: 8000/tcp

5.4 Front with an ALB

- Target group: HTTP 8000, health-check path */healthz*.
- Listener: HTTPS 443 with an ACM certificate. Redirect HTTP 80 → 443.
- Security group: only ALB SG can reach the ECS service SG.

5.5 ChromaDB in a container

ChromaDB's file-based persistence needs durable storage. Options:

- **EFS mount** on the ECS task — simplest, supports multiple replicas.
- **Run ChromaDB in server mode** on a separate EC2 / ECS task, using its HTTP API.
- **Migrate to OpenSearch Serverless** with k-NN index — fully managed (see §7).

6. Switching Ollama to Amazon Bedrock

Bedrock removes the need to run Ollama yourself, eliminates GPU costs, and gives you access to Claude, Llama, Mistral, and Titan models via a single API. Code changes in *src/agents/compliance_agent.py*:

```
# Before:
from langchain_ollama import ChatOllama
self.llm = ChatOllama(model=model_name, temperature=0.1)

# After (Bedrock):
from langchain_aws import ChatBedrock
self.llm = ChatBedrock(
    model_id="anthropic.claude-3-5-sonnet-20241022-v2:0",
    region_name=os.environ.get("AWS_REGION", "us-east-1"),
    model_kwargs={"temperature": 0.1, "max_tokens": 1024},
)
```

- Install: *pip install langchain-aws boto3*
- IAM: instance role needs *bedrock:InvokeModel* for the chosen model ARN.
- Enable the model in Bedrock → Model Access in the console (one-time).
- Update your *.env*: remove *OLLAMA_BASE_URL* / *OLLAMA_MODEL*, add *AWS_REGION=us-east-1* and *BEDROCK_MODEL_ID=<arn>*.

7. Managed services: ElastiCache, OpenSearch, S3

7.1 Amazon ElastiCache (Redis)

- Create a Redis cluster (cache.t4g.small is enough for sessions).
- Attach the cluster's security group to allow port 6379 from the API SG.
- Update `.env`: `REDIS_HOST=<primary-endpoint>`, `REDIS_PORT=6379`.
- Enable AUTH + TLS — then set `REDIS_TLS=true` and pass `ssl=True`, `password=<AUTH_TOKEN>` in `RedisMemory.__init__`.

7.2 Amazon OpenSearch Serverless (vector store)

Replace ChromaDB with managed k-NN search — recommended once you have > 100k vectors or need HA.

- Create an OpenSearch Serverless collection (type: *Vector search*).
- Create an index with an *hnsw* field matching your embedding dim (768 for mpnet).
- Write a thin adapter in `src/retrieval/opensearch_store.py` implementing the same interface as `VectorStore` (`add_documents`, `search`, `hybrid_search`, `get_collection_stats`). The rest of the app remains unchanged.

```
# Sketch:
from opensearchpy import OpenSearch, RequestsHttpConnection
from requests_aws4auth import AWS4Auth
import boto3

credentials = boto3.Session().get_credentials()
awsauth = AWS4Auth(credentials.access_key, credentials.secret_key,
                    os.environ["AWS_REGION"], "aoss",
                    session_token=credentials.token)
client = OpenSearch(hosts=[{"host": ENDPOINT, "port": 443}], http_auth=awsauth,
                    use_ssl=True, verify_certs=True, connection_class=RequestsHttpConnection)
```

7.3 S3 for document ingestion

- Upload raw regulation PDFs / TXTs to `s3://your-bucket/regulations/`
- The `/ingest` endpoint downloads via boto3 and passes to `DocumentLoader`.
- Enable S3 bucket versioning and Object Lock if you need an audit trail.

8. Security hardening

- **Secrets:** never commit `.env`. Store `API_TOKEN`, `REDIS_AUTH`, Bedrock model ARNs in Secrets Manager. Inject at container start via `secrets` block (ECS) or `aws secretsmanager` on EC2 boot.
- **IAM:** separate roles for ingestion (S3 read only) and runtime (Bedrock invoke, Redis SG, CloudWatch write). No `*:*` policies.
- **Network:** put API + data tier in private subnets; only the ALB sits in public subnets.
- **TLS everywhere:** ACM cert on ALB, internal traffic over VPC (still fine in HTTP within private subnets but prefer HTTPS).
- **Request limits:** in nginx / ALB, set body size 1 MB, rate-limit /query to 30 req/min per IP.
- **Input validation:** cap query length (e.g. 4 KB) in the Pydantic model.
- **Dependency pinning:** pin `requirements.txt` exact versions; scan with *pip-audit* or Amazon Inspector.
- **Patch cadence:** rebuild the image weekly; use EC2 Image Builder for AMI refresh.

9. Monitoring & observability

- **CloudWatch Logs:** agent streams stdout. Create a metric filter on "ERROR" → alarm → SNS → email/Slack.
- **CloudWatch Metrics:** expose latency + retrieval count via a Prometheus *prometheus-fastapi-instrumentator*, scrape with a sidecar or forward to CloudWatch.
- **ALB access logs** → S3, queryable via Athena.
- **Synthetics:** CloudWatch Synthetic canary hitting /readyz every minute.
- **Cost monitoring:** AWS Budgets alarm at 80% of expected monthly.

10. Cost estimates (us-east-1, on-demand, USD/month)

Rough ballparks; always confirm with the AWS Pricing Calculator.

Option / Component	Estimated monthly cost
Option A — g5.xlarge (GPU Ollama) 24/7	~\$730 (compute) + \$10 EBS + \$0 Redis local = ~\$740
Option A — c6i.2xlarge (CPU-only, no LLM)	~\$250 + \$10 EBS = ~\$260
Option A + Bedrock (Claude Sonnet)	~\$260 EC2 + usage-based (\$3/M input tokens)
Option B — Fargate 2 tasks (0.5 vCPU / 1 GB)	~\$35
Option B — ALB + data transfer	~\$20
Option B — OpenSearch Serverless (2 OCUs min)	~\$350
Option B — ElastiCache t4g.small	~\$22
Option B — Bedrock Claude Sonnet, 1M input tok/day	~\$90
S3 storage + egress (5 GB + 50 GB out)	~\$5
CloudWatch Logs (1 GB)	~\$1

Punch line: if you only need inference occasionally, Option B with Bedrock is cheapest in absolute terms and scales to zero Fargate load. If the LLM is hit constantly, a reserved g5.xlarge or Savings Plan often beats Bedrock per token.

11. Troubleshooting checklist

Symptom	Likely cause & fix
API returns 502 at the ALB	Target group unhealthy — check /healthz; ensure SG on tasks allows ALB SG; confirm the
"Found 0 results" in logs	Ingestion ran with a different EMBEDDING_MODEL than the API. Re-ingest after wiping chrom
Scores > 100%	Keyword / rerank boost applied without clamp. See vector_store.py hybrid_search and retriever
Redis connection refused	Security group on ElastiCache doesn't allow the API SG; or REDIS_HOST still pointing at 127.0
Ollama timeout on EC2	GPU drivers missing — use the Deep Learning AMI or install NVIDIA CUDA drivers. Con
Chunker hangs	Small sections trigger the overlap infinite loop. Apply the chunker fix (forward-progress guard) o
CORS error in browser	ALLOWED_ORIGIN in .env does not match the UI URL exactly (no trailing slash, include the sc
High Bedrock bill	Reduce top_k, trim retrieved_docs before sending to LLM, cache identical queries in Redis for 5